Train Llama2 with AWS Trainium on Amazon EKS

by Sanjeev Ganjihal and Scott Perry | on 29 JAN 2024 | in Amazon Elastic Kubernetes Service, Amazon Machine Learning, Artificial Intelligence, Compute, Containers, Generative AI, Technical How-to | Permalink

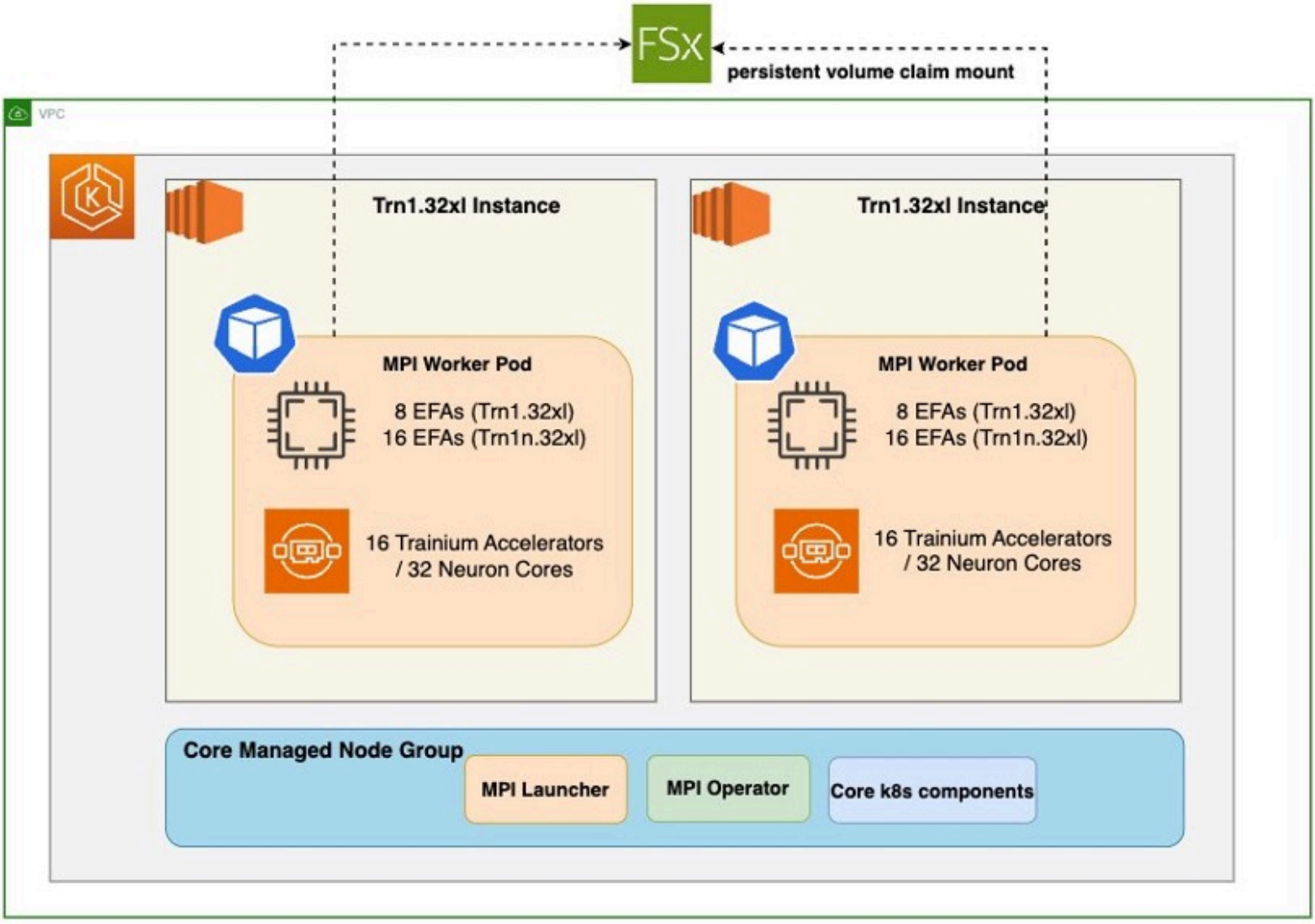
Introduction

Generative AI is not only transforming the way businesses function but also accelerating the pace of innovation within the broader AI field. This transformative force is redefining how businesses use technology, equipping them with capabilities to create human-like text, images, code, and audio, which were once considered beyond reach. Generative AI offers a range of applications that extend beyond simply executing prompts and facilitating interactive conversations. These models are increasingly used in diverse scenarios such as code generation, content summarization, data analysis, and more. Their adoption by enterprises across various industries highlights the versatility and utility of LLMs in addressing a multitude of complex tasks and challenges. However, these advantages pose a new set of challenges, particularly in the realms of training and operationalizing these massive models.

The escalating scale of Large Language Models (LLMs) and Generative AI greatly increases computational demands, leading to higher costs associated with the development and deployment. As the scale of data and the complexity of these models grow, so too does the need for more substantial resources to train them efficiently. This trend underscores the importance of cost-effective solutions like Amazon Elastic Kubernetes Service (Amazon EKS), which provides the necessary scalability and computational power to manage these extensive training workloads without incurring prohibitive expenses. According to projections by TIRIAS Research, AI infrastructure costs could surpass $76 billion by 2028. The existing business frameworks find it challenging to transfer these growing costs to consumers, necessitating either the advent of new business models or a substantial reduction in costs to ensure the continued growth and affordability of GenAI.

Amidst the rising costs and the increasingly scarce global compute supply, AWS Trainium offers a practical solution for model developers facing these challenges. By using AWS Trainium, developers can reduce the cost of training their models by up to 50%, while also optimizing performance in distributed training use cases. This makes AWS Trainium a valuable asset for those looking to manage expenses and improve efficiency in the realm of deep learning and model development. For more detailed information about AWS Trainium and its capabilities, you can visit the AWS Trainium product page for in-depth insights.

Distributed training architecture with AWS Trainium and Amazon EKS



The solution builds on a Data on Amazon EKS Terraform-based blueprint, which allows users to easily provision an Amazon EKS cluster along with a managed EKS nodegroup containing Amazon Elastic Compute Cloud (Amazon EC2) Trn1 instances. Each trn1.32xlarge instance has 16 AWS Trainium chips, which can be used for scalable, high-performance, and cost-effective model training. Within the nodegroup, the Trn1 instances are connected via high-speed, low-latency elastic fabric adapter (EFA) networking to enable the collective communications required during distributed training.

Each Llama training job is executed via Kubernetes pods using a container image that includes the Neuron SDK (the software stack for Trn1 instances) and the AWS Neuron Reference for NeMo Megatron – a fork of the open-source packages NeMo and Apex that have been adapted for use with OpenXLA and AWS Neuron. The combined software stack provides advanced training strategies and features including data, tensor, pipeline and sequence parallelism, selective activation checkpointing, and ZeRO-1 optimizer sharding.

The Kubernetes MPI Operator is used to coordinate distributed training across multiple pods, where each worker pod runs on a single trn1.32xlarge instance.

An Amazon FSx for Lustre shared filesystem is attached to the worker pods, providing a shared location to store the dataset, tokenizer files, Llama training scripts, training logs, compilation artifacts, and model checkpoints.

Solution Overview

Training Llama2 using AWS Trainium on Amazon EKS

Note: This post makes use of Meta's Llama tokenizer, which is protected by a user license that must be accepted before the tokenizer files can be downloaded. Please ensure that you have access to the Llama files by requesting access here.

Prerequisites:

- Amazon EC2 Instance (x86) or AWS Cloud9 instance (x86) – for both, please ensure you have at least 100GB+ of storage
- AWS Command Line Interface (AWS CLI) v2
- kubectl
- Git (Only for Amazon EC2 instance); The AWS Cloud9 comes with Git installed by default
- Docker
- Terraform
- Python, pip, jq, unzip

To install all the prerequisites on Amazon EC2, you can run this script.

Note: Remember to log out and log back in after running the prerequisite script to ensure all changes, particularly the docker group changes, are applied to your user account

Step 1: **Clone the data on EKS repository**

```
git clone https://github.com/awslabs/data-on-eks.git
```

Navigate to trainium-inferentia directory.

```
cd data-on-eks/ai-ml/trainium-inferentia
```

By default **MPI operator** is not installed and its set to **false**.

```
variable "enable_mpi_operator" {
  description = "Flag to enable the MPI Operator deployment"
  type        = bool
  default     = false
}
```

For this post, we will run the below export command to set environment variables.

NOTE: As of January1, 2024 AWS Trainium instances are only available in us-west-2, us-east-1, and us-east-2 Regions.

```
export TF_VAR_enable_mpi_operator=true
export TF_VAR_region=us-west-2
export TF_VAR_trn1_32xl_min_size=4
export TF_VAR_trn1_32xl_desired_size=4
```

Step 2: Run the install script to provision an Amazon EKS cluster with all the add-ons needed for the solution.

Note: Before you run the script, you can also change the cluster name based on your naming requirements.

```
./install.sh
```

```
Apply complete! Resources: 95 added, 1 changed, 0 destroyed.

Outputs:

configure_kubectl = "aws eks --region us-west-2 update-kubeconfig --name trn1-inf2-biyme"
SUCCESS: Terraform apply of all modules completed successfully
```

Step 3: Get access to Amazon EKS cluster as we will perform the following steps.

```
aws eks update-kubeconfig --region us-west-2 --name trainium-inferentia
```

Step 4: Navigate to examples/llama2 directory

```
cd examples/llama2/
```

Run the 1-llama2-neuronx-pretrain-build-image.sh script to build the **neuronx-nemo-megatron** container image and push the image into Amazon ECR.

When prompted for a Region, enter the Region in which you launched your Amazon EKS cluster (Step 1).

```
./1-llama2-neuronx-pretrain-build-image.sh
```

```
[ec2-user@ip-172-31-36-181 llama2]$ ./1-llama2-neuronx-pretrain-build-image.sh
Enter the ECR region: us-west-2
```

Note: The image building and pushing to Amazon ECR will approximately take ~10 minutes.

```
Pushing image to ECR
The push refers to repository [123255318457.dkr.ecr.us-west-2.amazonaws.com/neuronx_nemo]
b10f5beced88: Pushed
236dddacef52: Pushed
18ecea7975e9: Pushed
9b77b8660707: Pushed
9daf837bb0d3: Pushed
8ab51563dbe1: Pushed
f35e1711d9e3: Pushed
7e51ed87ace9: Pushed
dd9303bc9343: Pushed
cfa8cb2ae09e: Pushed
a1360aae5271: Pushed
latest: digest: sha256:0c1fe3b0dd46cc91cefdbab386d6a76fd46a28e22898935aa4687c69d5b838a0 size: 2642
[ec2-user@ip-172-31-36-181 llama2]$
```

Step 5: Access the shared Amazon FSx stoage.

To copy files to this storage, we'll first launch and connect to a CLI pod running the neuronx-nemo-megatron docker image that you created previously.

Run the following script to launch the CLI pod:

```
./2-launch-cmd-shell-pod.sh
```

```
[ec2-user@ip-172-31-36-181 llama2]$ ./2-launch-cmd-shell-pod.sh
Using container image 123255318457.dkr.ecr.us-west-2.amazonaws.com/neuronx_nemo:latest
pod/cli-cmd-shell created

NAME            READY   STATUS              RESTARTS   AGE
cli-cmd-shell   0/1     ContainerCreating   0          1s
cmd-shell       1/1     Running             0          103m
etcd-server     1/1     Running             0          103m
```

Run the following command to see the CLI pod going into 'Running' state:

```
kubectl get pod -w
```

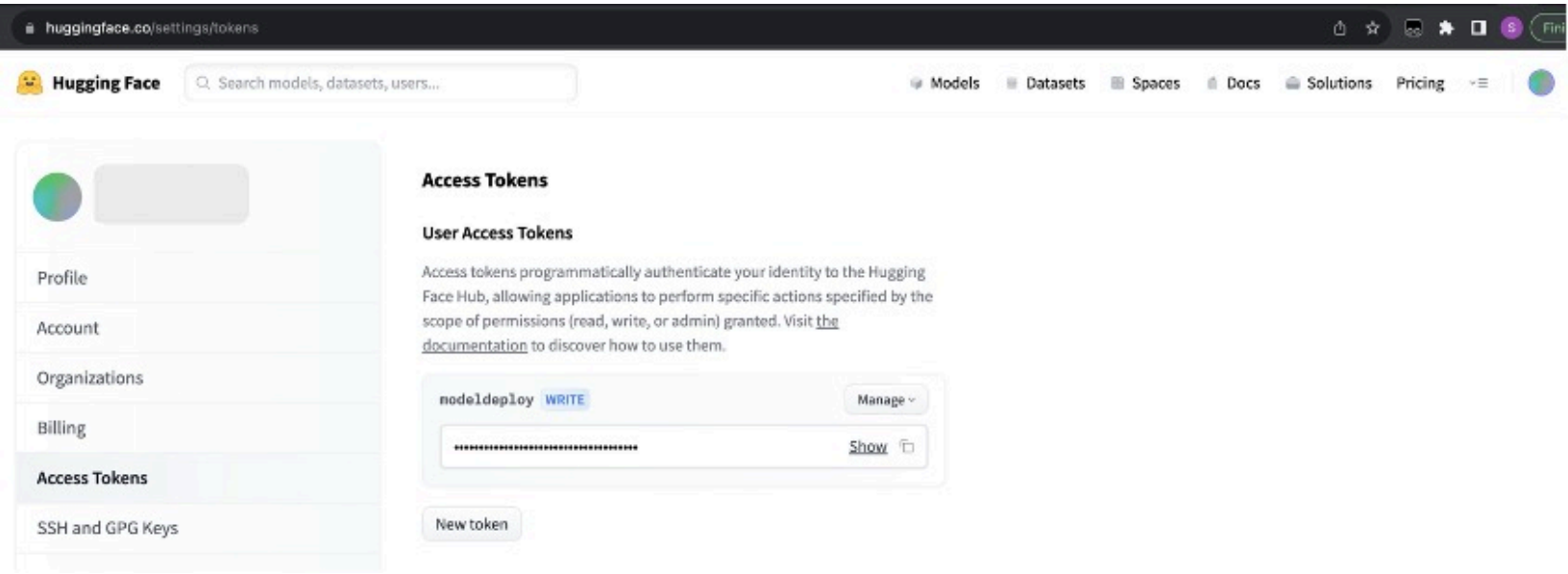
```
[ec2-user@ip-172-31-36-181 llama2]$ kubectl get po
NAME            READY   STATUS    RESTARTS   AGE
cli-cmd-shell   1/1     Running   0          69s
cmd-shell       1/1     Running   0          104m
etcd-server     1/1     Running   0          104m
```

Step 6: Once the CLI pod is '**Running**', connect to it using the following command:

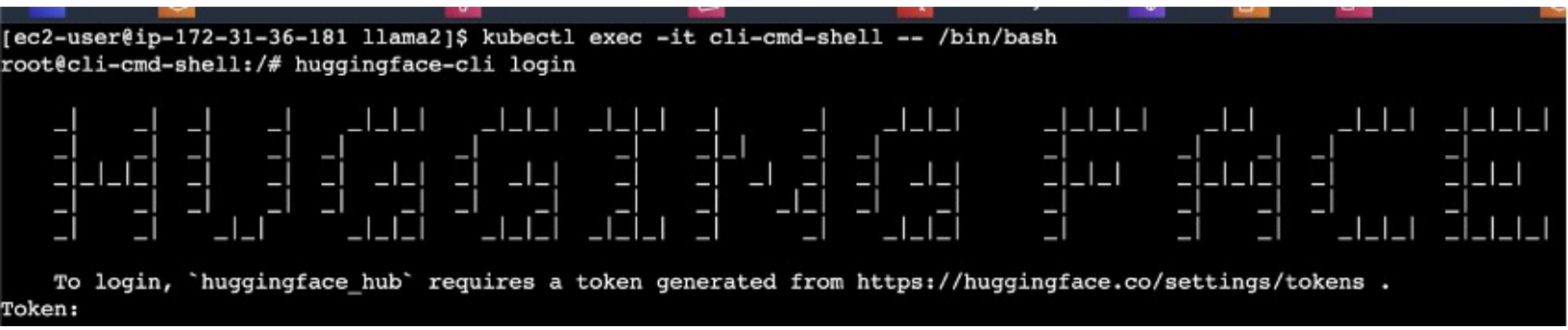
```
kubectl exec -it cli-cmd-shell -- /bin/bash
```

From the CLI pod, we'll download the Llama tokenizer files. First, run the huggingface-cli login command to login to Hugging Face using your access token.

The access token is found under **Settings** → **Access Tokens** on the Hugging Face website.




```
huggingface-cli login
```



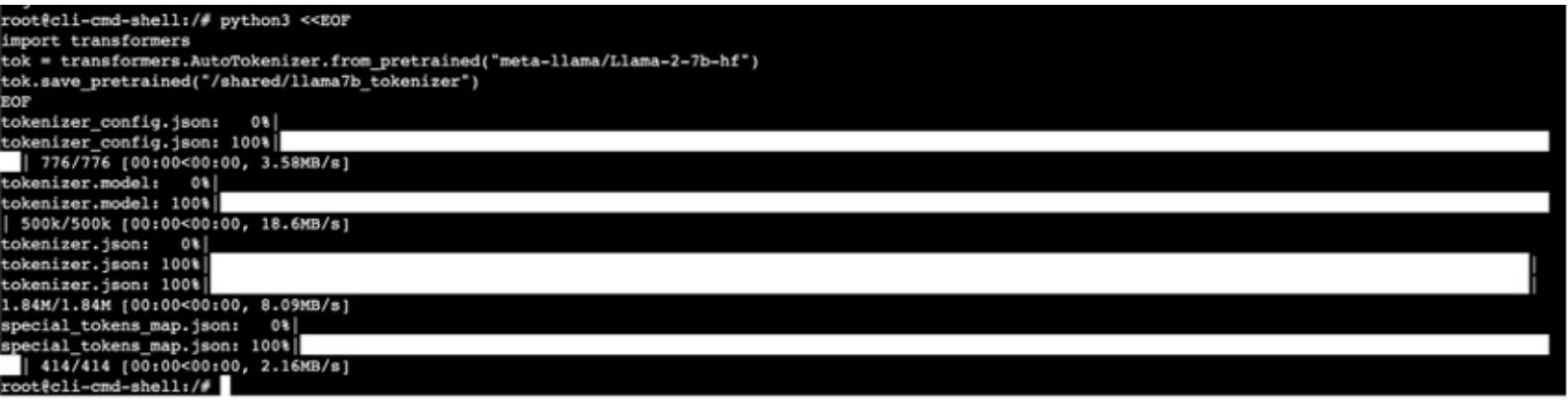
Paste the access token and hit enter.

Note: Do not add the token as a Git credential

```
Your token has been saved to /root/.cache/huggingface/token
Login successful
root@cli-cmd-shell:/#
```

Step 7: Download the llama7-7b tokenizer files to /shared/llama7b_tokenizer by running the python code.

```
python3 <<EOF
import transformers
tok = transformers.AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-hf")
tok.save_pretrained("/shared/llama7b_tokenizer")
EOF
```



Step 8: Download and tokenize the RedPajama-Data-1T-Sample dataset (a small subset of the full RedPajama dataset that contains 1B tokens).

While still connected to the CLI pod, use Git to download the dataset

```
cd /shared
git clone https://huggingface.co/datasets/togethercomputer/RedPajama-Data-1T-Sample \
    data/RedPajama-Data-1T-Sample
```

```
root@cli-cmd-shell:/# cd /shared
git clone https://huggingface.co/datasets/togethercomputer/RedPajama-Data-1T-Sample \
    data/RedPajama-Data-1T-Sample
Cloning into 'data/RedPajama-Data-1T-Sample'...
remote: Enumerating objects: 16, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 16 (delta 0), reused 0 (delta 0), pack-reused 14
Unpacking objects: 100% (16/16), 5.15 KiB | 155.00 KiB/s, done.
Filtering content: 100% (11/11), 5.04 GiB | 139.53 MiB/s, done.
root@cli-cmd-shell:/shared#
```

Step 9: Tokenize the dataset using the preprocessing script included with neuronx-nemo-megatron. This preprocessing step will take approximately 60 minutes to run on a trn1.32xl instance.

```
cd /shared

# Clone the neuronx-nemo-megatron repo, which includes the required scripts
git clone https://github.com/aws-neuron/neuronx-nemo-megatron.git

# Combine the separate redpajama files to a single jsonl file
cat /shared/data/RedPajama-Data-1T-Sample/*.jsonl > /shared/redpajama_sample.jsonl

# Run preprocessing script using llama tokenizer
python3 neuronx-nemo-megatron/nemo/scripts/nlp_language_modeling/preprocess_data_for_megatro
    --input=/shared/redpajama_sample.jsonl \
    --json-keys=text \
    --tokenizer-library=huggingface \
    --tokenizer-type=/shared/llama7b_tokenizer \
    --dataset-impl=mmap \
    --output-prefix=/shared/data/redpajama_sample \
    --append-eod \
    --need-pad-id \
```

```
root@cli-cmd-shell:/shared# cd /shared

# Clone the neuronx-nemo-megatron repo, which includes the required scripts
git clone https://github.com/aws-neuron/neuronx-nemo-megatron.git

# Combine the separate redpajama files to a single jsonl file
cat /shared/data/RedPajama-Data-1T-Sample/*.jsonl > /shared/redpajama_sample.jsonl

# Run preprocessing script using llama tokenizer
python3 neuronx-nemo-megatron/nemo/scripts/nlp_language_modeling/preprocess_data_for_megatron.py \
    --input=/shared/redpajama_sample.jsonl \
    --json-keys=text \
    --tokenizer-library=huggingface \
    --tokenizer-type=/shared/llama7b_tokenizer \
    --dataset-impl=mmap \
    --output-prefix=/shared/data/redpajama_sample \
    --append-eod \
    --need-pad-id \
    --workers=32
Cloning into 'neuronx-nemo-megatron'...
remote: Enumerating objects: 3947, done.
remote: Counting objects: 100% (396/396), done.
remote: Compressing objects: 100% (264/264), done.
remote: Total 3947 (delta 240), reused 228 (delta 132), pack-reused 3551
Receiving objects: 100% (3947/3947), 51.34 MiB | 6.08 MiB/s, done.
Resolving deltas: 100% (1081/1081), done.
Updating files: 100% (3144/3144), done.
```

```
Using cls_token, but it is not set yet.
Using pad_token, but it is not set yet.
Using mask_token, but it is not set yet.
[NeMo I 2023-12-15 03:06:04 auto_tokenizer:171] 1 special tokens added, resize your model accordingly.
Using mask_token, but it is not set yet.
Using sep_token, but it is not set yet.
Using cls_token, but it is not set yet.
Using pad_token, but it is not set yet.
Using mask_token, but it is not set yet.
[NeMo I 2023-12-15 03:06:04 auto_tokenizer:171] 1 special tokens added, resize your model accordingly.
Using mask_token, but it is not set yet.
Processed 100 documents (1.7292996070742144 docs/s, 0.09564705239363401 MB/s).
Processed 200 documents (3.3359380768148816 docs/s, 0.1946840406442424 MB/s).
Processed 300 documents (4.687307084468494 docs/s, 0.2794042272198859 MB/s).
Processed 400 documents (5.50457094972834 docs/s, 0.32773132690297135 MB/s).
Processed 500 documents (6.51161170332192 docs/s, 0.3807085330985887 MB/s).
Processed 600 documents (7.3035634877013385 docs/s, 0.44001332052065467 MB/s).
Processed 700 documents (8.249375928721566 docs/s, 0.5036391305777852 MB/s).
Processed 800 documents (9.080257772301826 docs/s, 0.5445893856059639 MB/s).
Processed 900 documents (9.938422918613735 docs/s, 0.5966223305927177 MB/s).
Processed 1000 documents (10.441935899858965 docs/s, 0.6250654624460327 MB/s).
Processed 1100 documents (11.104159687759433 docs/s, 0.6622396273031164 MB/s).
Processed 1200 documents (11.787166683341772 docs/s, 0.7026800834969645 MB/s).
Processed 1300 documents (12.443421049031251 docs/s, 0.7480264696532214 MB/s).
Processed 1400 documents (13.04801301882044 docs/s, 0.7798618947171307 MB/s).
Processed 1500 documents (13.673609682246518 docs/s, 0.8062790106843506 MB/s).
```

```
Processed 930100 documents (285.14587262236824 docs/s, 1.5728584867061932 MB/s).
Processed 930200 documents (285.17571694767327 docs/s, 1.5729533232683988 MB/s).
Processed 930300 documents (285.2055889745344 docs/s, 1.5730834215756024 MB/s).
Processed 930400 documents (285.2353185659166 docs/s, 1.57320680461184 MB/s).
Processed 930500 documents (285.2651123092018 docs/s, 1.573340482885587 MB/s).
root@cli-cmd-shell:/shared#
```

As you can see, 930500 documents are processed as part of data tokenization process.

Note: When we later launch our training jobs in Amazon EKS, the training pods will run the training script from within neuronx-nemo-megatron/nemo/examples directory on FSx. This is convenient, because it will let you modify your training script directly on Amazon FSx without requiring that you rebuild the neuronx-nemo-megatron container for every change.

Step 10: Modify the **test_llama** script /shared/neuronx-nemo-megatron/nemo/examples/nlp/language_modeling/test_llama.sh to update the following two lines. These lines tell the training pod workers where to find the Llama tokenizer and the dataset on the Amazon FSx filesystem.

Run:

```
sed -i 's#^\(: ${TOKENIZER_PATH=\).*#\1/shared/llama7b_tokenizer}#' /shared/neuronx-nemo-megatr
sed -i 's#^\(: ${DATASET_PATH=\).*#\1/shared/data/redpajama_sample_text_document}#' /shared/neu
```

Before:

```
: ${TOKENIZER_PATH=$HOME/llamav2_weights/7b-hf}
: ${DATASET_PATH=$HOME/examples_datasets/llama_7b/book.jsonl-processed_text_document}
```

After:

```
: ${TOKENIZER_PATH=/shared/llama7b_tokenizer}
: ${DATASET_PATH=/shared/data/redpajama_sample_text_document}
```

Step 11: When you are finished with the CLI pod you can delete it by running:

```
kubectl delete pod cli-cmd-shell
```

```
[ec2-user@ip-172-31-36-181 llama2]$ kubectl get po
NAME             READY   STATUS    RESTARTS   AGE
cli-cmd-shell    1/1     Running   0          92m
cmd-shell        1/1     Running   0          3h15m
etcd-server      1/1     Running   0          3h15m
[ec2-user@ip-172-31-36-181 llama2]$ kubectl delete po cli-cmd-shell
pod "cli-cmd-shell" deleted
[ec2-user@ip-172-31-36-181 llama2]$
```

Step 12: We are now ready to launch our pre-compilation and training jobs!

Before we can run the **training** job, we first need to run a pre-compilation job in order to prepare the model artifacts. This step extracts and compiles the underlying compute graphs for the Llama2-7B model and generates AWS Neuron executable files (NEFFs) that can run on the **AWS Trainium** chips. These NEFFs are stored in a persistent AWS Neuron cache on Amazon FSx so that the training job can later access them.

Before you run the compilation job make sure **MPI operator** is functional by running this command:

```
kubectl get all -n mpi-operator
```

```
[ec2-user@ip-172-31-36-181 llama2]$ kubectl get all -n mpi-operator
NAME                                 READY   STATUS    RESTARTS   AGE
pod/mpi-operator-6b8447f5cd-4x97x    1/1     Running   0          34s

NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/mpi-operator   1/1     1            1           34s

NAME                                      DESIRED   CURRENT   READY   AGE
replicaset.apps/mpi-operator-6b8447f5cd   1         1         1       34s
[ec2-user@ip-172-31-36-181 llama2]$
```

Run the pre-compilation script:

```
./3-llama2-neuronx-mpi-compile.sh
```

```
[ec2-user@ip-172-31-36-181 llama2]$ ./3-llama2-neuronx-mpi-compile.sh
Using container image 123255318457.dkr.ecr.us-west-2.amazonaws.com/neuronx_nemo:latest
mpijob.kubeflow.org/test-mpi-compile created

NAME                                READY   STATUS              RESTARTS   AGE
cmd-shell                           1/1     Running             0          3h46m
etcd-server                         1/1     Running             0          3h47m
test-mpi-compile-launcher-15db8     0/1     Init:0/1            0          0s
test-mpi-compile-worker-0           0/1     ContainerCreating   0          0s
test-mpi-compile-worker-1           0/1     ContainerCreating   0          0s
test-mpi-compile-worker-2           0/1     ContainerCreating   0          0s
test-mpi-compile-worker-3           0/1     ContainerCreating   0          0s
[ec2-user@ip-172-31-36-181 llama2]$
```

```
[ec2-user@ip-172-31-36-181 llama2]$ kubectl get po  | grep compile
test-mpi-compile-launcher-15db8   1/1     Running   0          85s
test-mpi-compile-worker-0         1/1     Running   0          85s
test-mpi-compile-worker-1         1/1     Running   0          85s
test-mpi-compile-worker-2         1/1     Running   0          85s
test-mpi-compile-worker-3         1/1     Running   0          85s
[ec2-user@ip-172-31-36-181 llama2]$
```

Pre-compilation will take approximately **10 minutes** when using 4 trn1.32xlarge nodes.

Periodically run kubectl get pods | grep compile and wait until you see that the compile job shows **Completed**.

```
[ec2-user@ip-172-31-36-181 llama2]$ kubectl get po | grep compile
test-mpi-compile-launcher-j9c41   0/1     Completed   0
[ec2-user@ip-172-31-36-181 llama2]$
```

Step 13: When pre-compilation is complete, you can then launch the pre-training job on 4 trn1.32xl nodes by running the following script:

```
./4-llama2-neuronx-mpi-train.sh
```

Step 14: To monitor the training job output, first, find the name of the launcher pod associated with your training job:

```
kubectl get pods | grep launcher
```

```
[ec2-user@ip-172-31-36-181 llama2]$ kubectl get pods | grep launcher
test-mpi-compile-launcher-j9c4l   0/1     Completed   0          78m
test-mpi-train-launcher-g52f4     1/1     Running     0          4m21s
[ec2-user@ip-172-31-36-181 llama2]$
```

Once you have identified the name of the launcher pod and see that it is Running, the next step is to determine its **UID**.

Replace **test-mpi-train-launcher-xxx** with your launcher pod name in the following command and it will output the UID:

```
kubectl get pod test-mpi-train-launcher-g52f4 -o json | jq -r ".metadata.uid"
```

```
[ec2-user@ip-172-31-36-181 llama2]$ kubectl get pod test-mpi-train-launcher-g52f4 -o json | jq -r ".metadata.uid"
7eb8879b-a85a-4ff0-bd9d-34e7ae19f1e2
```

Step 15: Use the **UID** to determine the log path so you can tail the training logs. Replace UID with the previous value.

```
kubectl exec -it test-mpi-train-worker-0 -- tail -f /shared/nemo_experiments/<UID>/0/log
```

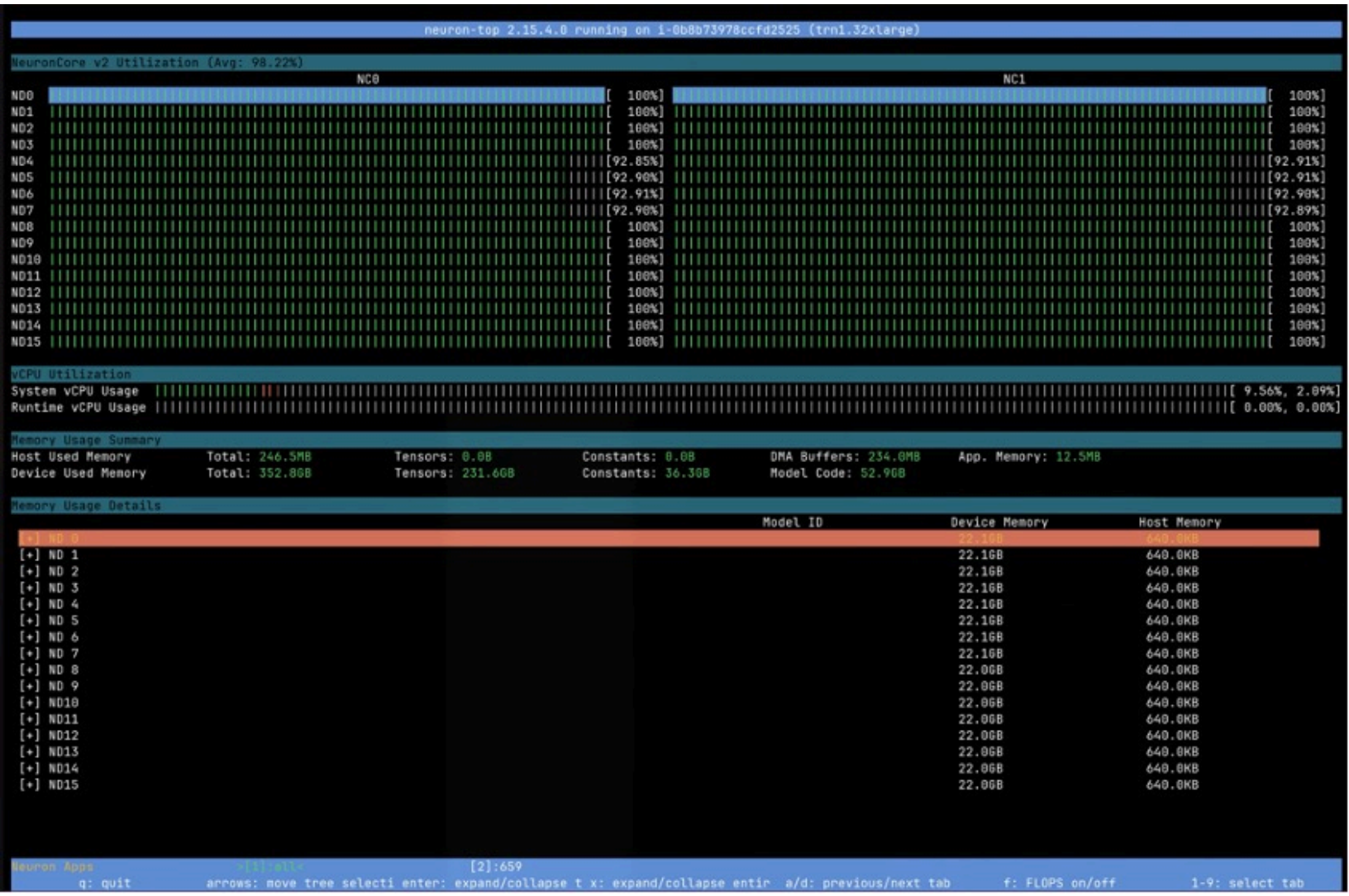
```
Epoch 0:   0%|          | 29/20101 [14:21<165:40:23, 29.71s/it, loss=9.15, v_num=, val_loss_step=11.20,
Epoch 0:   0%|          | 30/20101 [14:37<163:06:18, 29.26s/it, loss=9.15, v_num=, val_loss_step=11.20,
Epoch 0:   0%|          | 30/20101 [14:37<163:06:20, 29.26s/it, loss=8.82, v_num=, val_loss_step=11.20,
Epoch 0:   0%|          | 31/20101 [14:53<160:42:23, 28.83s/it, loss=8.82, v_num=, val_loss_step=11.20,
Epoch 0:   0%|          | 31/20101 [14:53<160:42:24, 28.83s/it, loss=8.62, v_num=, val_loss_step=11.20,
Epoch 0:   0%|          | 32/20101 [15:09<158:27:06, 28.42s/it, loss=8.62, v_num=, val_loss_step=11.20,
Epoch 0:   0%|          | 32/20101 [15:09<158:27:07, 28.42s/it, loss=8.47, v_num=, val_loss_step=11.20,
Epoch 0:   0%|          | 33/20101 [15:25<156:20:14, 28.05s/it, loss=8.47, v_num=, val_loss_step=11.20,
Epoch 0:   0%|          | 33/20101 [15:25<156:20:16, 28.05s/it, loss=8.34, v_num=, val_loss_step=11.20,
```

When you are done viewing the logs, you can press CTRL-C to quit the tail command.

Step 16: To monitor AWS Trainium chip utilization you can use the neuron-top command.

Neuron-top is a console-based tool for monitoring AWS Neuron and system-related performance metrics on trn1/inf2/inf1 instances. You can launch neuron-top on one of the worker pods as follows:

```
kubectl exec -it test-mpi-train-worker-0 -- /bin/bash -l neuron-top
```

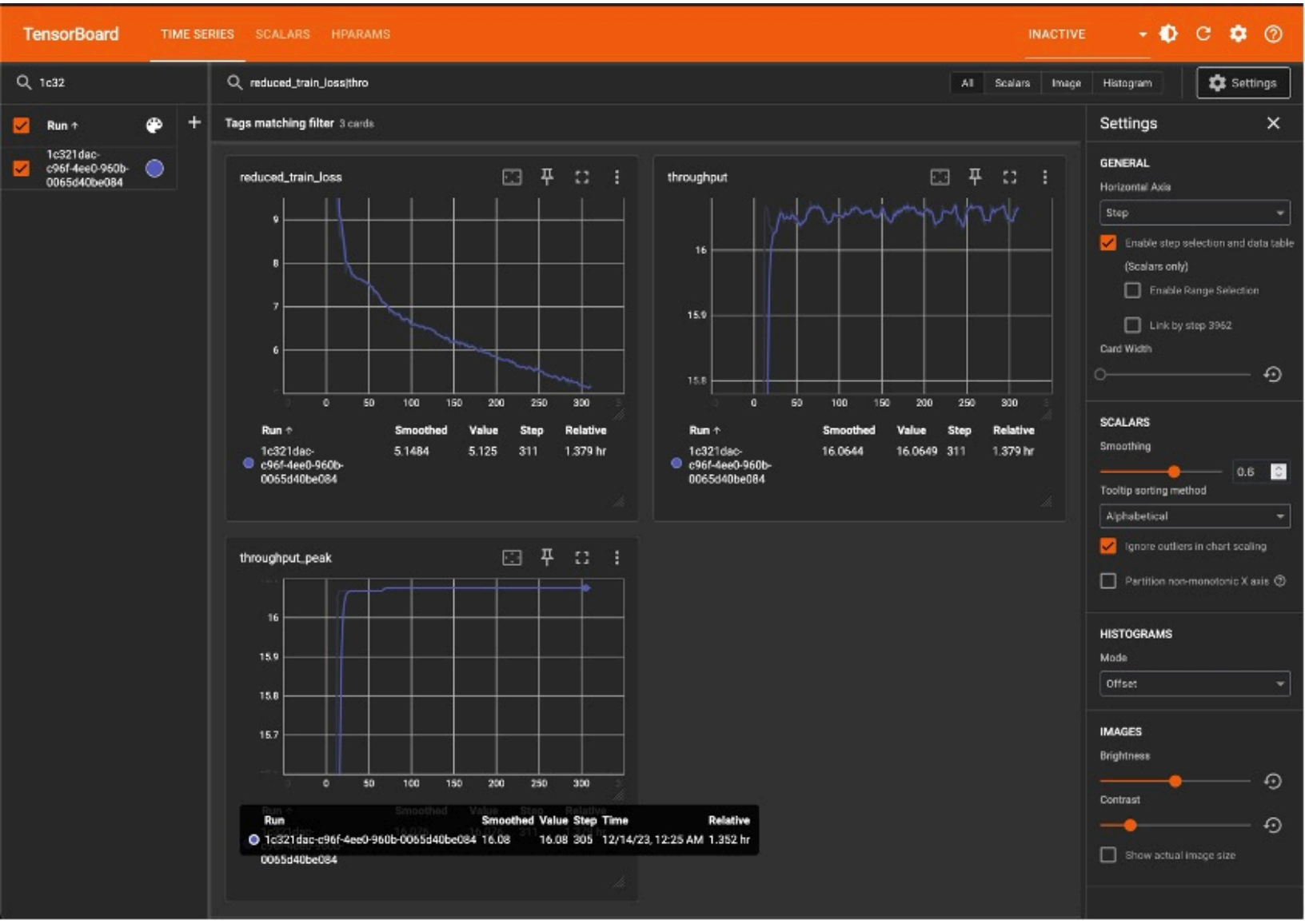


Step 17: Create a Tensorboard deployment to visualize these logs by running the following command:

```
./5-deploy-tensorboard.sh
```

Tensorboard logs are also available in the /shared/nemo_experiments/ directory on the Amazon FSx for Lustre filesystem. Once the deployment is ready the script will output a password-protected URL for your new Tensorboard deployment.

Load balancer URL will display as output after running the shell script. Open the loadbalancer URL in the browser to view your training progress:



Cleaning up

To clean up all the provisioned resources for this post, run the cleanup script:

```
cd data-on-eks/ai-ml/trainium-inferentia
./cleanup.sh
```

Conclusion

In this post we showed you how AWS Trainium's integration with Neuronx-nemo-megatron on Amazon EKS marks a significant stride in tackling the rising computational demands and cost challenges in training advanced AI models. Notably, AWS Trainium offers up to 50% cost savings in training, coupled with high-performance capabilities. This, along with the Neuron SDK's compatibility with popular machine learning (ML) frameworks, establishes an optimal environment for AI model training. The inclusion of the MPI Operator and Data on Amazon EKS (DoEKS) further enhances the efficiency and scalability of distributed training. Innovative features like ZeRO-1 optimizer sharding and selective activation checkpointing not only make cutting-edge ML research more accessible but also drive the AI industry towards unprecedented innovation.

Key links & references

AWS Trainium

AWS Neuron

AWS Neuron Reference for NeMo Megatron

Tensor, Pipeline, Sequence, Data parallelism

ZeRO-1 (Optimizer Sharding)

Activation Checkpointing

MPI Operator

DataOnEKS (DoEKS)